If your function receives double the number of requests (200 requests per second), but only requires half the time to process each request (250 ms), then the concurrency is still 50:

```
Concurrency = (200 requests/second) * (0.25 second/request) = 50
```

Test your understanding of concurrency

Suppose you have a function that takes, on average, 200 ms to run. During peak load, you observe 5,000 requests per second. What is the concurrency of your function during peak load?

Answer

The average function duration is 200 ms, or 0.2 seconds. Using the concurrency formula, you can plug in the numbers to get a concurrency of 1,000:

```
Concurrency = (5,000 requests/second) * (0.2 seconds/request) = 1,000
```

Alternatively, an average function duration of 200 ms means that your function can process 5 requests per second. To handle the 5,000 request per second workload, you need 1,000 execution environment instances. Thus, the concurrency is 1,000:

```
Concurrency = (5,000 requests/second) / (5 requests/second) = 1,000
```

Understanding reserved concurrency and provisioned concurrency

By default, your account has a concurrency limit of 1,000 concurrent executions across all functions in a Region. Your functions share this pool of 1,000 concurrency on an on-demand basis. Your functions experience throttling (that is, they start to drop requests) if you run out of available concurrency.

Some of your functions might be more critical than others. As a result, you might want to configure concurrency settings to ensure that critical functions get the concurrency that they need. There are two types of concurrency controls available: reserved concurrency and provisioned concurrency.

- Use **reserved concurrency** to set both the maximum and minimum number of concurrent instances to reserve a portion of your account's concurrency for a function. This is useful if you

don't want other functions taking up all the available unreserved concurrency. When a function has reserved concurrency, no other function can use that concurrency.

- Use **provisioned concurrency** to pre-initialize a number of environment instances for a function. This is useful for reducing cold start latencies.

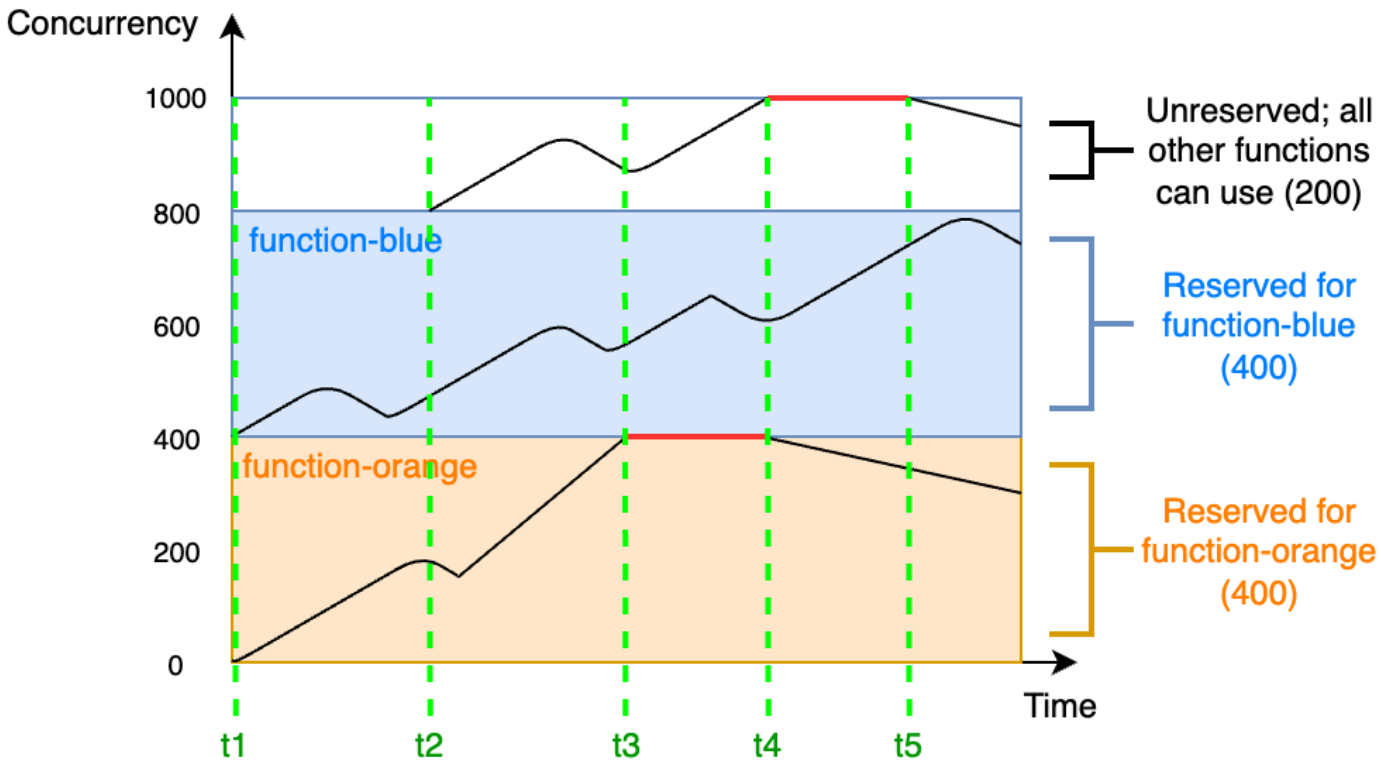
Reserved concurrency

If you want to guarantee that a certain amount of concurrency is available for your function at any time, use reserved concurrency.

Reserved concurrency sets the maximum and minimum number of concurrent instances that you want to allocate to your function. When you dedicate reserved concurrency to a function, no other function can use that concurrency. In other words, setting reserved concurrency can impact the concurrency pool that's available to other functions. Functions that don't have reserved concurrency share the remaining pool of unreserved concurrency.

Configuring reserved concurrency counts towards your overall account concurrency limit. There is no charge for configuring reserved concurrency for a function.

To better understand reserved concurrency, consider the following diagram:



In this diagram, your account concurrency limit for all the functions in this Region is at the default limit of 1,000. Suppose you have two critical functions, `function-blue` and `function-orange`, that routinely expect to get high invocation volumes. You decide to give 400 units of reserved concurrency to `function-blue`, and 400 units of reserved concurrency to `function-orange`. In this example, all other functions in your account must share the remaining 200 units of unreserved concurrency.

The diagram has five points of interest:

- At `t1`, both `function-orange` and `function-blue` begin receiving requests. Each function begins to use up its allocated portion of reserved concurrency units.
- At `t2`, `function-orange` and `function-blue` steadily receive more requests. At the same time, you deploy some other Lambda functions, which begin receiving requests. You don't allocate reserved concurrency to these other functions. They begin using the remaining 200 units of unreserved concurrency.
- At `t3`, `function-orange` hits the max concurrency of 400. Although there is unused concurrency elsewhere in your account, `function-orange` cannot access it. The red line indicates that `function-orange` is experiencing throttling, and Lambda may drop requests.
- At `t4`, `function-orange` starts to receive fewer requests and is no longer throttling. However, your other functions experience a spike in traffic and begin throttling. Although there is unused concurrency elsewhere in your account, these other functions cannot access it. The red line indicates that your other functions are experiencing throttling.
- At `t5`, other functions start to receive fewer requests and are no longer throttling.

From this example, notice that reserving concurrency has the following effects:

- **Your function can scale independently of other functions in your account.** All of your account's functions in the same Region that don't have reserved concurrency share the pool of unreserved concurrency. Without reserved concurrency, other functions can potentially use up all of your available concurrency. This prevents critical functions from scaling up if needed.
- **Your function can't scale out of control.** Reserved concurrency caps your function's maximum and minimum concurrency. This means that your function can't use concurrency reserved for other functions, or concurrency from the unreserved pool. Additionally, reserved concurrency acts as both a lower and upper bound - it reserves the specified capacity exclusively for your function while also preventing it from scaling beyond that limit. You can reserve concurrency

to prevent your function from using all the available concurrency in your account, or from overloading downstream resources.

- **You may not be able to use all of your account's available concurrency.** Reserving concurrency counts towards your account concurrency limit, but this also means that other functions cannot use that chunk of reserved concurrency. If your function doesn't use up all of the concurrency that you reserve for it, you're effectively wasting that concurrency. This isn't an issue unless other functions in your account could benefit from the wasted concurrency.

To learn how to manage reserved concurrency settings for your functions, see Configuring reserved concurrency for a function.

Provisioned concurrency

You use reserved concurrency to define the maximum number of execution environments reserved for a Lambda function. However, none of these environments come pre-initialized. As a result, your function invocations may take longer because Lambda must first initialize the new environment before being able to use it to invoke your function. When Lambda has to initialize a new environment in order to carry out an invocation, this is known as a cold start. To mitigate cold starts, you can use provisioned concurrency.

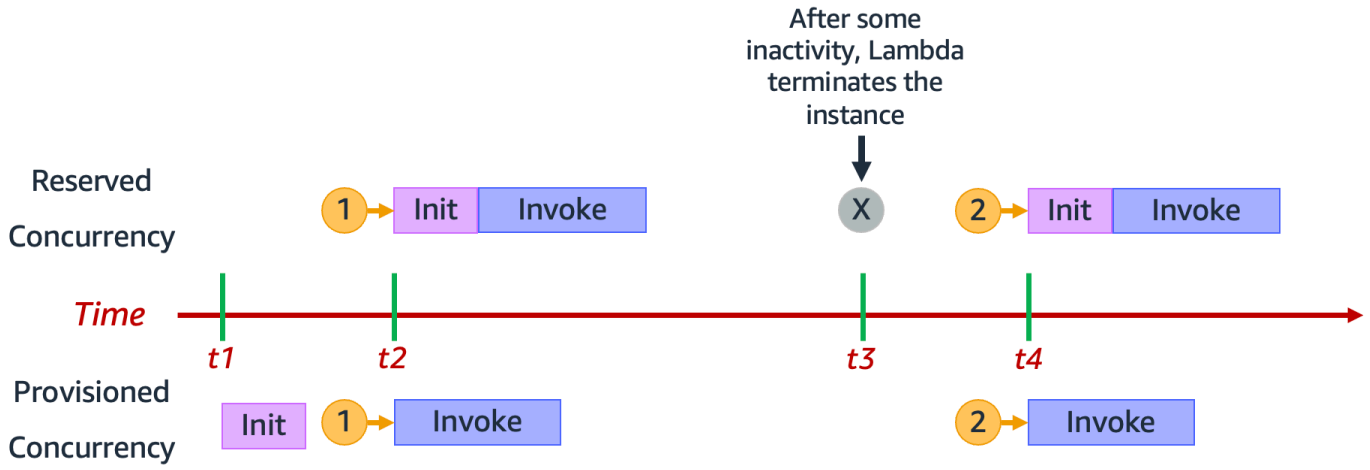
Provisioned concurrency is the number of pre-initialized execution environments that you want to allocate to your function. If you set provisioned concurrency on a function, Lambda initializes that number of execution environments so that they are prepared to respond immediately to function requests.

> **Note**
>
> Using provisioned concurrency incurs additional charges to your account. If you're working with the Java 11 or Java 17 runtimes, you can also use Lambda SnapStart to mitigate cold start issues at no additional cost. SnapStart uses cached snapshots of your execution environment to significantly improve startup performance. You cannot use both SnapStart and provisioned concurrency on the same function version. For more information about SnapStart features, limitations, and supported Regions, see Improving startup performance with Lambda SnapStart.

When using provisioned concurrency, Lambda still recycles execution environments in the background. For example, this can occur after an invocation failure. However, at any given time,

Lambda always ensures that the number of pre-initialized environments is equal to the value of your function's provisioned concurrency setting. Importantly, even if you're using provisioned concurrency, you can still experience a cold start delay if Lambda has to reset the execution environment.

In contrast, when using reserved concurrency, Lambda may completely terminate an environment after a period of inactivity. The following diagram illustrates this by comparing the lifecycle of a single execution environment when you configure your function using reserved concurrency compared to provisioned concurrency.

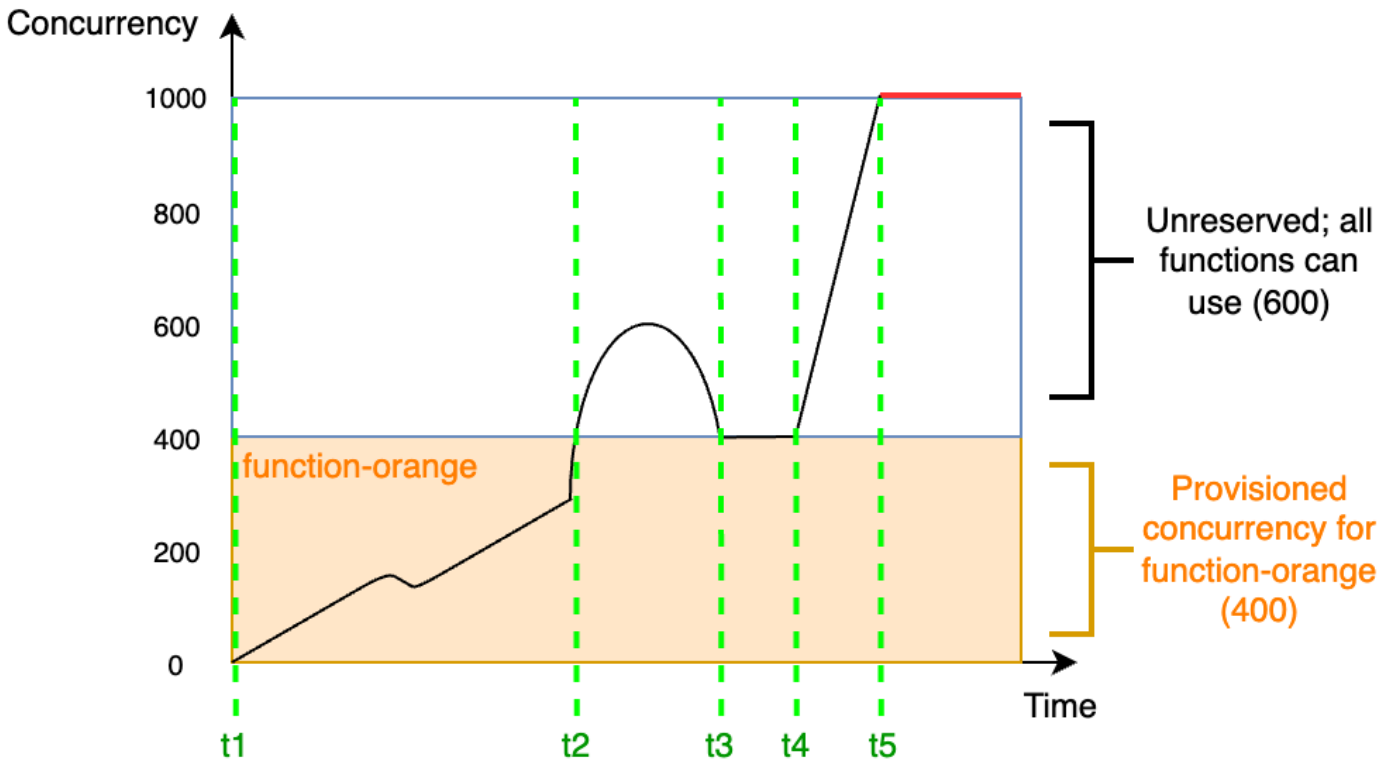


The diagram has four points of interest:

Time	Reserved concurrency	Provisioned concurrency
t1	Nothing happens.	Lambda pre-initializes an execution environment instance.
t2	Request 1 comes in. Lambda must initialize a new execution environment instance.	Request 1 comes in. Lambda uses the pre-initialized environment instance.
t3	After some inactivity, Lambda terminates the active environment instance.	Nothing happens.

Time	Reserved concurrency	Provisioned concurrency
t4	Request 2 comes in. Lambda must initialize a new execution environment instance.	Request 2 comes in. Lambda uses the pre-initialized environment instance.

To better understand provisioned concurrency, consider the following diagram:



In this diagram, you have an account concurrency limit of 1,000. You decide to give 400 units of provisioned concurrency to `function-orange`. All functions in your account, *including* `function-orange`, can use the remaining 600 units of unreserved concurrency.

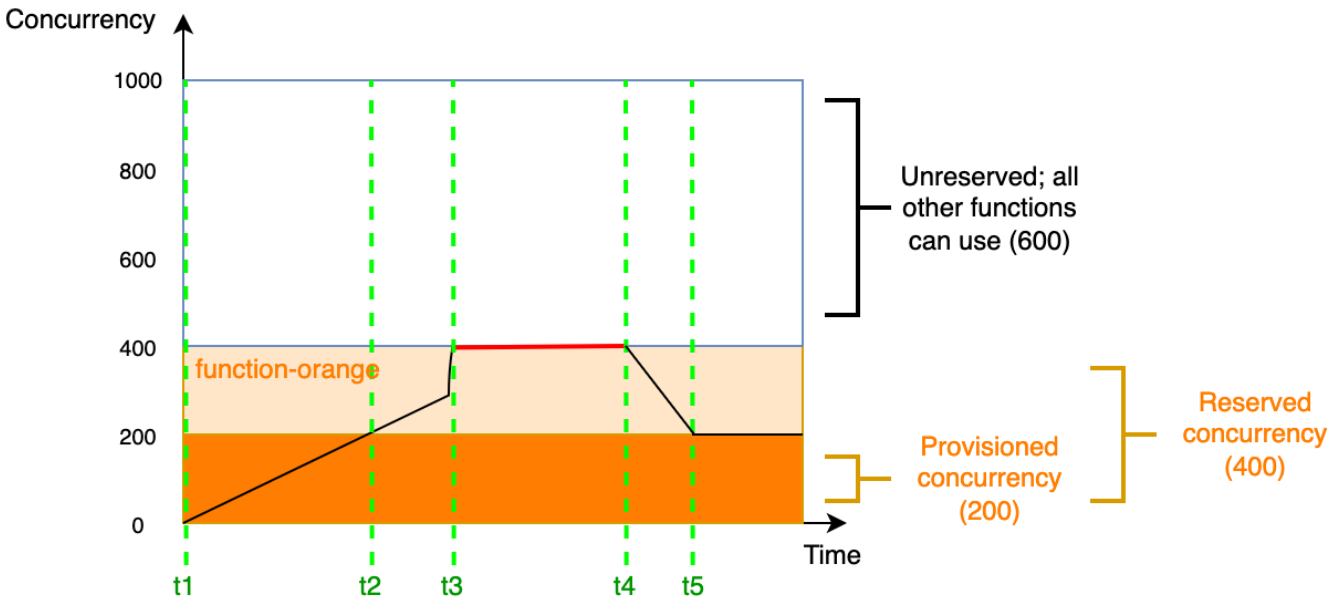
The diagram has five points of interest:

- At `t1`, `function-orange` begins receiving requests. Since Lambda has pre-initialized 400 execution environment instances, `function-orange` is ready for immediate invocation.
- At `t2`, `function-orange` reaches 400 concurrent requests. As a result, `function-orange` runs out of provisioned concurrency. However, since there's still unreserved concurrency available,

Lambda can use this to handle additional requests to `function-orange` (there's no throttling). Lambda must create new instances to serve these requests, and your function may experience cold start latencies.

- At `t3`, `function-orange` returns to 400 concurrent requests after a brief spike in traffic. Lambda is again able to handle all requests without cold start latencies.
- At `t4`, functions in your account experience a burst in traffic. This burst can come from `function-orange` or any other function in your account. Lambda uses unreserved concurrency to handle these requests.
- At `t5`, functions in your account reach the maximum concurrency limit of 1,000, and experience throttling.

The previous example considered only provisioned concurrency. In practice, you can set both provisioned concurrency and reserved concurrency on a function. You might do this if you had a function that handles a consistent load of invocations on weekdays, but routinely sees spikes of traffic on weekends. In this case, you could use provisioned concurrency to set a baseline amount of environments to handle request during weekdays, and use reserved concurrency to handle the weekend spikes. Consider the following diagram:



In this diagram, suppose that you configure 200 units of provisioned concurrency and 400 units of reserved concurrency for `function-orange`. Because you configured reserved concurrency, `function-orange` cannot use any of the 600 units of unreserved concurrency.

This diagram has five points of interest:

- At `t1`, `function-orange` begins receiving requests. Since Lambda has pre-initialized 200 execution environment instances, `function-orange` is ready for immediate invocation.
- At `t2`, `function-orange` uses up all its provisioned concurrency. `function-orange` can continue serving requests using reserved concurrency, but these requests may experience cold start latencies.
- At `t3`, `function-orange` reaches 400 concurrent requests. As a result, `function-orange` uses up all its reserved concurrency. Since `function-orange` cannot use unreserved concurrency, requests begin to throttle.
- At `t4`, `function-orange` starts to receive fewer requests, and no longer throttles.
- At `t5`, `function-orange` drops down to 200 concurrent requests, so all requests are again able to use provisioned concurrency (that is, no cold start latencies).

Both reserved concurrency and provisioned concurrency count towards your account concurrency limit and Regional quotas. In other words, allocating reserved and provisioned concurrency can impact the concurrency pool that's available to other functions. Configuring provisioned concurrency incurs charges to your AWS account.

> **Note**
>
> If the amount of provisioned concurrency on a function's versions and aliases adds up to the function's reserved concurrency, then all invocations run on provisioned concurrency. This configuration also has the effect of throttling the unpublished version of the function (`$LATEST`), which prevents it from executing. You can't allocate more provisioned concurrency than reserved concurrency for a function.

To manage provisioned concurrency settings for your functions, see Configuring provisioned concurrency for a function. To automate provisioned concurrency scaling based on a schedule or application utilization, see Using Application Auto Scaling to automate provisioned concurrency management.

How Lambda allocates provisioned concurrency

Provisioned concurrency doesn't come online immediately after you configure it. Lambda starts allocating provisioned concurrency after a minute or two of preparation. For each function,

Lambda can provision up to 6,000 execution environments every minute, regardless of AWS Region. This is exactly the same as the concurrency scaling rate for functions.

When you submit a request to allocate provisioned concurrency, you can't access any of those environments until Lambda completely finishes allocating them. For example, if you request 5,000 provisioned concurrency, none of your requests can use provisioned concurrency until Lambda completely finishes allocating the 5,000 execution environments.

Comparing reserved concurrency and provisioned concurrency

The following table summarizes and compares reserved and provisioned concurrency.

Topic	Reserved concurrency	Provisioned concurrency
Definition	Maximum number of execution environment instances for your function.	Set number of pre-provisioned execution environment instances for your function.
Provisioning behavior	Lambda provisions new instances on an on-demand basis.	Lambda pre-provisions instances (that is, before your function starts receiving requests).
Cold start behavior	Cold start latency possible, since Lambda must create new instances on-demand.	Cold start latency not possible, since Lambda doesn't have to create instances on-demand.
Throttling behavior	Function throttled when reserved concurrency limit reached.	If reserved concurrency not set: function uses unreserved concurrency when provisioned concurrency limit reached. If reserved concurrency set: function throttled when reserved concurrency limit reached.